

Qwirkle Documentation

Contents

1	Introduction	2
2	Project files	4
2.1	global_variables.py	4
2.2	utilities.py	5
2.2.1	Functions for debugging purposes	5
2.2.2	Non-debugging utilities	5
3	Board extraction	6
3.1	Connect	6
3.1.1	Transformations	6
3.1.2	Corner detection	8
3.1.3	Perspective warping	9
3.2	Bonus	9
3.2.1	Transformations	9
3.2.2	Corner detection	11
3.2.3	Perspective warping	11
4	Obtaining the shape templates for template matching	12
4.1	Connect	12
4.2	Bonus	14
5	Initial board configuration	16
5.1	Connect	16
5.1.1	Locating and classifying the shaped pieces	16
5.1.2	Locating and classifying the number pieces	18
5.2	Bonus	19
5.2.1	Locating and classifying the shaped pieces	19
6	Detecting moves	21
6.1	Connect	21
6.2	Bonus	24
7	Calculating the score	26

1 Introduction

In this report, I will be discussing a strategy for implementing a program in Python with OpenCV that aims to analyze camera-taken photos of a board game being played, calculating the scores and determining the moves that are performed.

The board game to be analyzed is known as **Qwirkle Connect**. I will also be exploring the variant of **Qwirkle Classic**, which is played without a board. General information about the game, such as what it looks like, how it is played, what the pieces do, what moves are legal, how the scores are calculated, and other basic prerequisites will not be presented in this document.

A Qwirkle Connect game consists of 21 images: the first image, indexed 00, shows the initial configuration of the board. The other images, labeled 01 to 20, depict the moves that are being played. A Qwirkle Classic game consists of 20 images: the first image, indexed 01, shows the first move, which is also the initial configuration of the board. The other images, labeled 02 to 20, depict the moves that are being played.



Initial configuration (Connect).



Final move (Connect).



First move (Classic).



Final move (Classic).

The actual goal of the project is to analyze such images by calculating the positions, colors, and shapes of the pieces being placed, and also computing the overall score of the move. Each image (except for the initial configurations of the Qwirkle Connect games, labeled 00) denotes a move of which a detailed description must be obtained. When a

piece is placed on the board, the following features must be specified:

- **Position:** row and column. For Connect, the rows range from 1 to 16, and the columns take values from A to P . For Classic, the origin $(0,0)$ belongs to the top-left-most piece, and the positions are calculated with respect to this piece.
- **Color:** a piece may be white, red, orange, yellow, green, or blue.
- **Shape:** a piece can be a circle, square, cross, diamond, 4-pointed star, or 8-pointed star.

Once this information is worked out for every single placed piece, the final score of the move must also be returned. The training data consisted of five Qwirkle Connect matches and one Qwirkle Classic match; since a Connect game corresponds to 21 different images and a Classic game corresponds to 20, that equals to 125 training images. The "fake test" was composed of only one game for each variant, so 41 images in total. The image files were provided in the *.jpg* format and the labels corresponding to the moves were *.txt* files that followed a certain format.

I divided my solution into a few components, each having its own detailed discussion in its dedicated subchapter. A brief presentation of the steps would be as follows:

1. **Board extraction:** as can be seen in the images presented earlier, the photos were taken at an angle, and there are redundant background elements. To significantly reduce the complexity of the task at hand, the input images must be processed to keep everything horizontal and exclude any unnecessary attributes.
2. **Initial board configuration:** once the images have been transformed and the boards have been extracted, the first image is analyzed. As a result, the initial configuration of the board is stored in a configuration matrix. The elements have values corresponding to empty cells, number pieces (for Connect), and shaped pieces (consisting of a unique color identifier and a unique shape identifier).
3. **Move detection:** when a move is performed, the image is compared with the previous image, and the differences are computed, highlighting the added pieces, calculating the score, and updating the configuration matrix accordingly. This process is applied from the first move (image 01) to the last move (image 20).

To implement these three main functionalities, I divided them into multiple subtasks and split the code into 7 separate python files. A general overview of the files:

- **global_variables.py:** constants which are referenced throughout the whole project (such as *WIDTH_BOARD*, *HEIGHT_CELL*, etc.). More details in *Section 2.1*.
- **utilities.py:** many useful functions that are referenced throughout various parts of the project (functions such as *resize_image*, *show_image*, *save_image*, *draw_contours*, and others). More details in *Section 2.2*.
- **board_extraction.py:** code for extracting the board, as mentioned in the first step. More details in *Section 3*.
- **hardcoded_template_maker.py:** imperative for template matching; creates templates. More details in *Section 4*. I didn't include this file in the project, since the code is not required to be able to run the project.
- **piece_classification.py:** classifies a shaped piece in terms of color and shape. More details in *Section 4* (obtaining the templates) and *Section 5* (classifying the pieces).
- **get_board_configuration.py:** contains the necessary functions to get the initial board configuration and then detect the moves. More details in *Section 5* (getting the initial board configuration) and *Section 6* (detecting moves).
- **calculate_score.py:** when pieces are added, the score is calculated. More details in *Section 7*.
- **run_project.py:** runs the project. More details on how to make sure the project runs can be found in *Section 8*.

2 Project files

2.1 global_variables.py

1. **BONUS:** boolean variable. Specifies whether the project is running for Qwirkle Classic (*BONUS == True*) or Connect.
2. **DATA_READ_PATH, BOARDS_SAVE_PATH, MOVES_SAVE_PATH:** specifies the directory paths for the input data, solutions, saving the extracted boards, and saving the highlighted detected moves. The path for the detected moves can be set to *None*, if one does not wish to save them.
3. **CONFIG_ROWS, CONFIG_COLUMNS:** the amount of rows and columns for the configuration matrix; 16x16 for Connect, 215x215 for Classic. The choice of matrix dimensions for the Classic variant stems from the following idea: if you place a piece, you have 107 pieces left that can go all the way to the left, or all the way to the right (same concept vertically). Thus, $107 * 2 + 1 = 215$. This is not realistic, since the bounds of this matrix will never be reached in any Qwirkle Classic game, but no project is perfect.
4. **CONFIG_CENTER_ROW, CONFIG_CENTER_COL:** necessary for the Classic variant. The origin point is the center of the matrix, which is (107, 107).
5. **WIDTH_BOARD, HEIGHT_BOARD:** the original images are of 3072x4080 dimensions. When extracting the board for Connect, I decided to set the final dimensions at 2816x2816, plus the additional pixels from extending the corners. The aim was to make the board as symmetric as possible, so the width and height are the same; furthermore, they are divisible by 16, which makes calculations simpler (the board has 16x16 cells) and they are not greater than the image's original dimensions. For Classic, the final dimensions are exactly 2816x2816.
6. **EXTEND_CORNERS:** the pieces on the last/first row/column slightly go over the board's edge in the Connect variant. Thus, extracting the board by focusing on the rectangle defined by the board's four corners would moderately crop those pieces. This variable defines the amount of pixels to extend the board's corners by; the drawback is that some additional redundant background will be included in the final image, but this solves the issue of cropped pieces. Ultimately, $WIDTH_BOARD = HEIGHT_BOARD = 2816 + 2 * EXTEND_CORNERS$.
7. **WIDTH_CELL, HEIGHT_CELL:** for **Connect**: the board is divided into 256 cells, on a 16x16 grid. Each cell is of approximately equal dimensions. The approximate width of a cell would be the width of the board divided by the number of cells on the width, which is 16. The approximate height of a cell would be the height of the board divided by the number of cells on the height. The final formula is $WIDTH_CELL = HEIGHT_CELL = (WIDTH_BOARD - 2 * EXTEND_CORNERS) / 16 = (HEIGHT_BOARD - 2 * EXTEND_CORNERS) / 16$.
For **Classic**: more details in *Section 5.2.1*.
8. **X_ORIGIN, Y_ORIGIN:** more details in *Section 5.2.1*.
9. **FREE_CELLS:** empty cells are cells where shaped pieces can be placed. In Qwirkle Connect, these are either empty cells or cells with number pieces in them, so $FREE_CELLS = \{ "0", "1", "2" \}$; in Classic, these are just empty cells, so $FREE_CELLS = \{ "0" \}$.
10. **BOARD_EXTRACTION_DISPLAYS, BOARD_EXTRACTION_SAVES:** there are certain intermediary filters and transformations applied to the original image when extracting the board. These variables indicate which of those transformations to display or save on disk throughout the process, for debugging purposes.
11. **HARDCODED_COLOR_RANGES:** color ranges for *cv.inRange*.
12. **COLORS, SHAPES:** all possible colors (6) and shapes (6) are stored in tuples.

2.2 utilities.py

2.2.1 Functions for debugging purposes

- **show_image(title, img):** displays an image.
- **save_image(img, path, file_name):** saves an image.
- **draw_board_corners(img, corners):** given an image and a list of four point coordinates, the function draws red circles at those coordinates.
- **check_display_and_save(img, img_name, save_dir, displays, saves, key):** throughout the initial process of extracting the board, the function checks which intermediary transformations will be displayed or saved on disk.
- **draw_contours(img, contours, color):** given an image, a list of contours and a drawing color, it draws the contours onto the image and returns the changed image.

2.2.2 Non-debugging utilities

- **get_contours(mask, list_type, min_area=0, max_area=float("inf"), hw_diff_thresh=float("inf"), min_length=0):** given a binary image and a list type (which is generally *cv.RETR_LIST* for all contours or *cv.RETR_EXTERNAL* for bigger exterior contours), this function finds the contours of the image and filters them according to a minimum acceptable area, a maximum area, an upper bound for the absolute difference between the width and height of the contour, as well as a minimum number of points making up the contour.
- **resize_image(img, new_w, new_h):** given an image, it resizes its dimensions to match the newly given width and height. Dimensions may be resized to be larger or smaller.
- **extend_piece(amount, x_piece, y_piece, w_piece, h_piece):** given the exact X and Y coordinates of a piece on the board and its dimensions, the function finds the maximum number of pixels by which it can extend the piece in all directions with respect to the edges of the image (the number will be smaller or equal to the argument *amount*) and updates the dimensions and coordinates of the piece, returning the new values.
- **get_piece_outline(board, board_name, x_piece, y_piece, w_piece, h_piece, erode=True):** for a shaped piece given as input, it finds the exact bounding rectangle that perfectly matches the shaped piece's form.
- **save_move(board_name, changes, score, game_index, move_index):** saves the detected moves and score in a file.

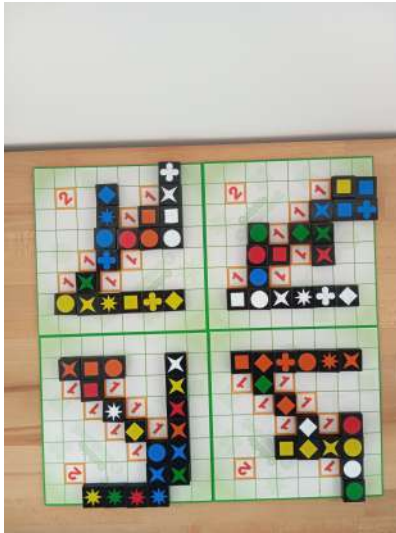
3 Board extraction

There are three main steps when it comes to extracting the board from the original image:

1. **Transformations:** applying image processing techniques to eliminate noise and redundant details.
2. **Corner detection:** locating the exact coordinates of the corners of the board.
3. **Warping the perspective:** the detected corners must match the default coordinates in the resulting image, such as $(0,0)$ for top-left.

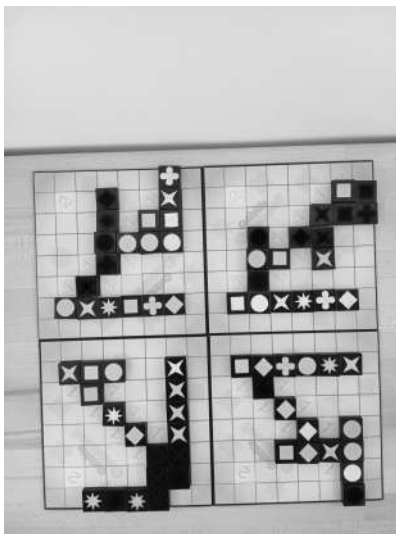
3.1 Connect

3.1.1 Transformations



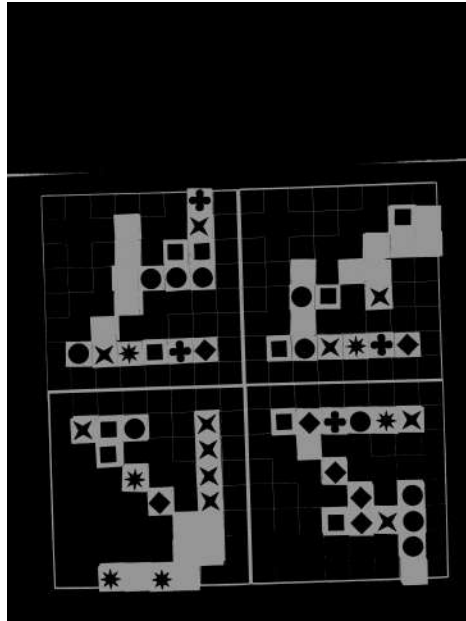
Original image (train data, game 4, move 20)

As can be seen, the outline of the board is **green**. Applying the `cv.split` function on the image will return three separate grayscale images, each representing the intensity of one primary color: B (blue), G (green), and R (red). Isolating the R channel will make the green pixels have very low intensity values, making them look dark.



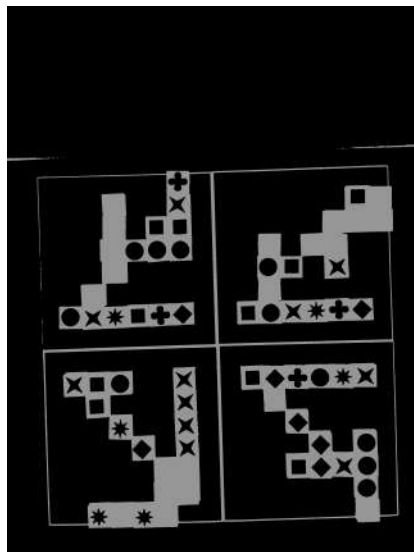
Taking the red channel

A threshold can be applied to obtain a binary image in which these dark pixels are the area of interest. Any pixel in the image with a threshold lower than 100 will be colored with white, whereas the others will be colored with black.

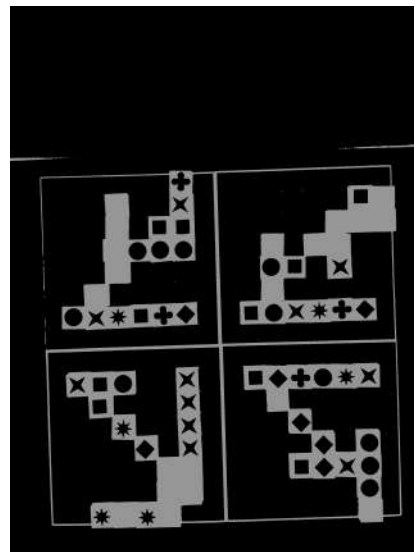


Thresholding

There may be some faint lines or salt-and-pepper-type noise, so applying a median blur of size 7 will get rid of most of the noise. The outline of the board will still look slightly washed, so dilating the image once with a kernel size of 3×3 should make the outline more prominent.

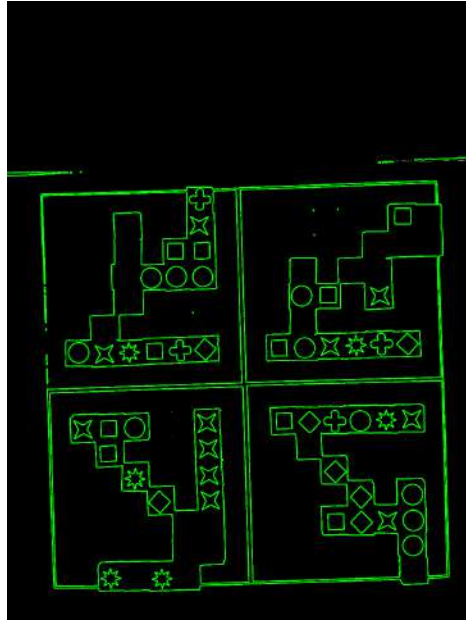


Median blur of kernel size 7.



One dilation of kernel size 3.

3.1.2 Corner detection



Contours of the final binary image

For each contour that consists of more than 3 points, the corners of the contour are found: the top-left corner is the point that minimizes the value of $x + y$, the top-right corner is the one the point that minimizes the value of $x - y$, the bottom-right corner is the point that maximizes the value of $x + y$, and the bottom-left corner is the point that maximizes the value of $x - y$.

Once these points have been determined, their area is calculated. If it is greater than the maximum area calculated so far, then those points become the new contour; the new "corners" of the board. Iterating through every detected contour and applying this logic will result in finding the corners of the board. They are then extended by the constant value *EXTEND_CORNERS*, which is 50 in this case.

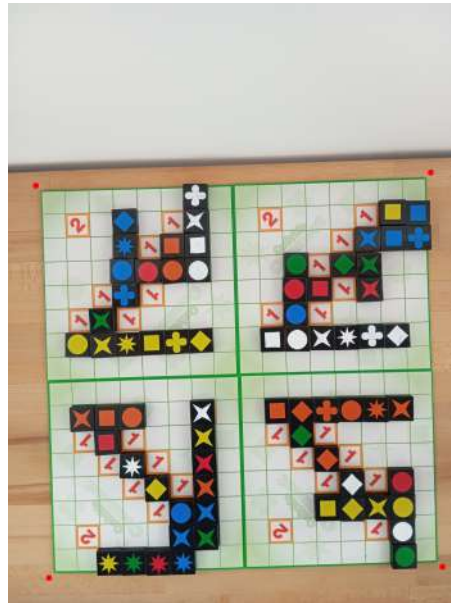


Figure 3: Detected corners, extended by 50 pixels in all directions

3.1.3 Perspective warping

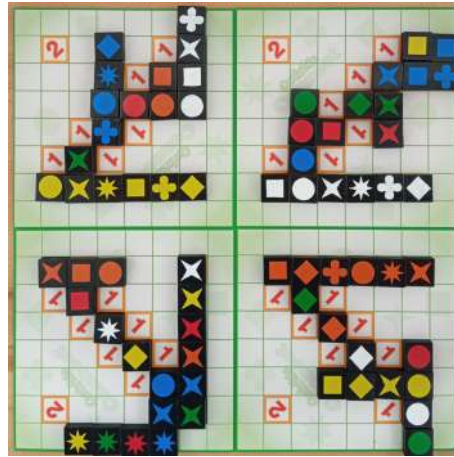


Figure 4: Result (perspective warping)

The detected corners are mapped to a set of default coordinates corresponding to the general corners of any image. The corners become $(0, 0)$, $(WIDTH_BOARD, 0)$, $(WIDTH_BOARD, HEIGHT_BOARD)$, $(0, HEIGHT_BOARD)$. OpenCV functions are applied, defining the transformation matrix and applying the warp.

3.2 Bonus

3.2.1 Transformations



Figure 5: Original image (train data, game 1, move 20)

Since there is no game board, the pieces can be spread all throughout the picture. The safest move would be to keep the whole table (brown area) and get rid of the white area from above the table. This can be done by isolating the white color from the image.

The BGR color space is not ideal for color section, since a change in lighting (brightness) drastically changes the R, G, and B values. The HSV color space separates color (hue) from brightness (value) and purity (saturation). The first step is to convert the image to the HSV color space.

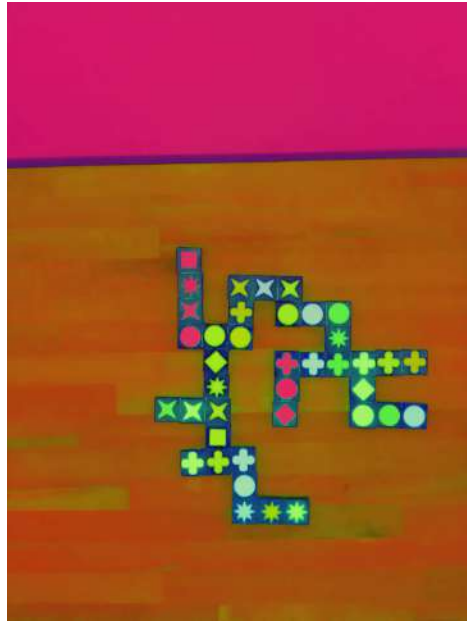


Figure 6: Conversion to HSV

A range can be defined for the white color: a lower bound and an upper bound for all three HSV values. Any pixel that fails to fall within that specific range is set to black. These bounds were found through experimenting with the *hsv_trackbar.py* file.

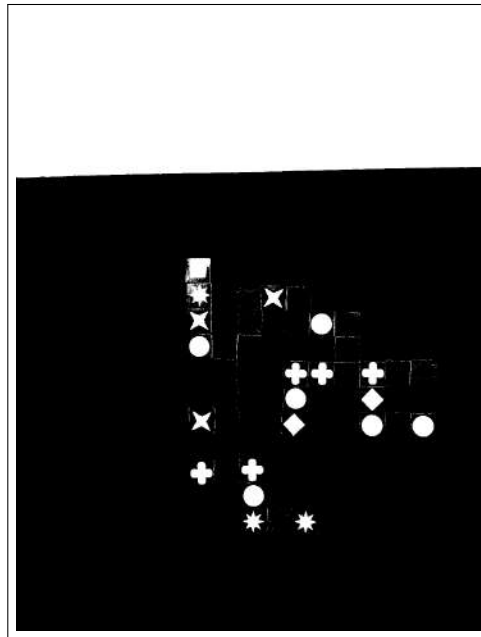


Figure 7: Applying *cv.inRange* to the HSV image to isolate the white color

3.2.2 Corner detection



Figure 8: Highlighted corners of the playing area

The last white pixel on the first column of the image should represent the top-left corner. The last white pixel on the last column should represent the top-right corner. Those pixels are approximately at the limit between the white area and the table area. They represent fine approximations for the corners of the playing area. The bottom-left and bottom-right corners are simply the bottom-left and bottom-right pixels of the image.

3.2.3 Perspective warping

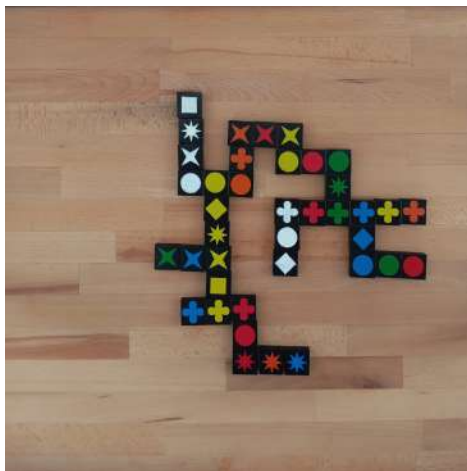


Figure 9: Result (perspective warping)

The process is identical to the perspective warping in the *Connect* version (**Section 3.1.3**).

4 Obtaining the shape templates for template matching

4.1 Connect

In order to apply template matching, there must be templates to match with. The file *hardcoded_template_maker.py* contains the necessary code to generate the templates for the shaped pieces.

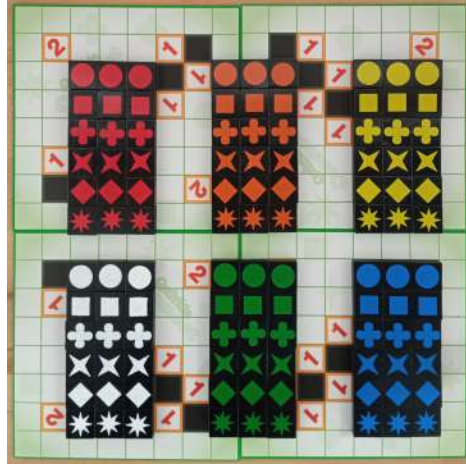


Figure 10: Source image for template creation

Firstly, I picked an image from the "aux" folder containing all 108 pieces displayed on the board. I hardcoded the areas of interest (6 perfect rectangles containing each set of pieces and nothing else).

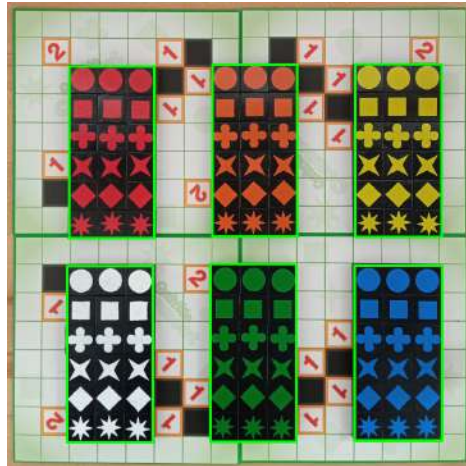


Figure 11: Hardcoded coordinates for the rectangles containing the sets of pieces

For each set, the image within the rectangle is converted to HSV. A lower-upper range for the color of the set is applied to obtain a binary image containing the shapes of the pieces.

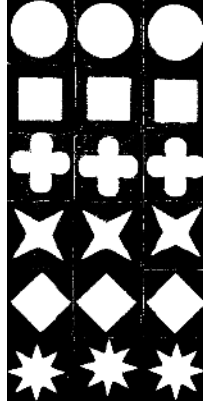


Figure 12: Mask for the red piece set

Contours of an area within $[3000, 25000]$ are obtained. That is a safe and general interval for detecting the shapes.

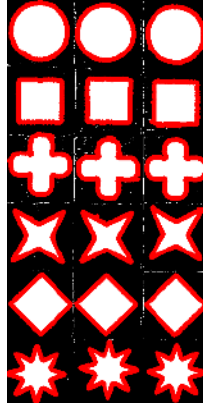


Figure 13: Contours of the mask of the red piece set

For each contour, the coordinates (x, y) and dimensions (w, h) of the corresponding bounding rectangle are obtained. The bounding rectangle is extracted from the mask, providing the entire shape. Any salt-and-pepper noise is cleaned up through a median blur of kernel size 7. The image is resized to 200×200 and stored in a list.

The first element of the list is a sublist containing all the resulting binary circle shapes. The second element is a sublist containing the crosses, and so on. Finally, the elements of each sublist are combined to provide a final template for each shape, which is then cleaned up with a median blur of kernel size 11.

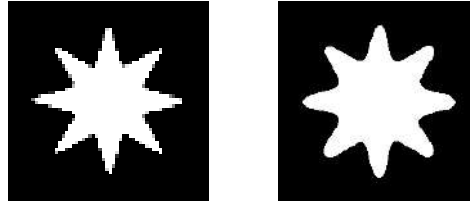


Figure 14: An element of the 8-pointed star sublist and the final 8-pointed star template

4.2 Bonus



Figure 15: Source image for template creation

As a source image to extract the shapes from, I picked the last image from the bonus training set. I converted the image to HSV, extracted the black color and obtained the contours. Only the contours of an area within [5000, 20000] made the cut (just some general values), as to get only the shapes.

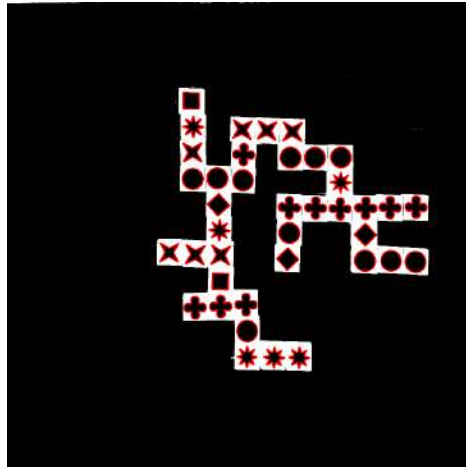


Figure 16: Contours of the shaped pieces

To double check and ensure that the shape of each piece is obtained in its entirety, I iterated through the contours, getting the bounding rectangle of each contour, extending its dimensions by 50 pixels in all directions using *extend_piece* from *utilities.py*, and using that new enlarged window to select the perfect outline of the piece from the source image, with help of *get_piece_outline* from *utilities.py*.

The piece is converted to HSV. The black color is extracted and the image is negated, so that the shape becomes white. A median blur of kernel size 7 is applied to get rid of any salt-and-pepper noise. The image is resized to 200x200. It is stored in its corresponding sublist, and then the same procedure is applied as for *Connect*: the elements of each sublist are combined and then cleaned up with another median blur.

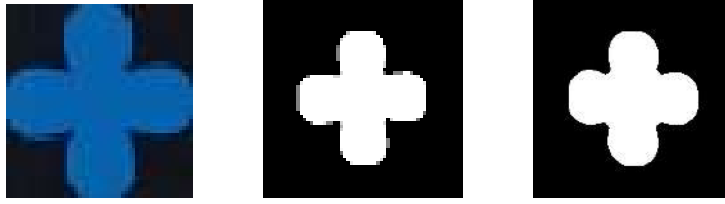


Figure 17: On the left is an example of perfect piece extraction. The middle image depicts the transformed image, added to the cross sublist. On the right is the final cross template.

5 Initial board configuration

5.1 Connect

5.1.1 Locating and classifying the shaped pieces

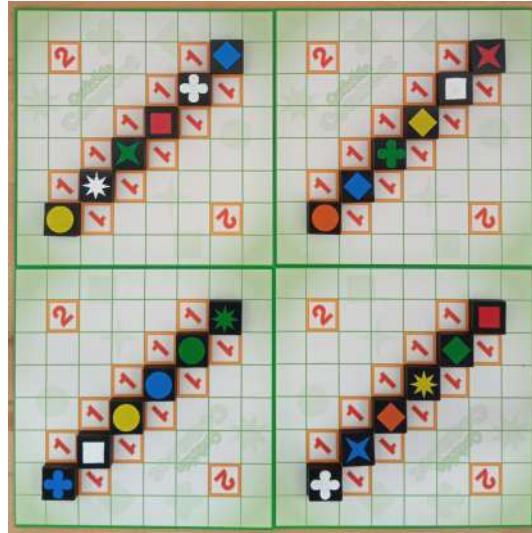


Figure 18: Source image

The above image is the initial configuration of the first training game. In order to isolate the shaped pieces, the black color is extracted from the HSV version of the image, and then the image is negated.

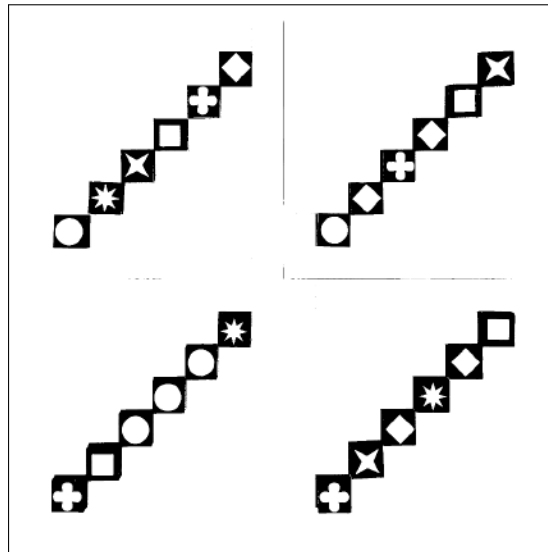


Figure 19: Mask of the shaped pieces

Erosion is applied a few times to completely entrap the shape inside the black rectangle, isolating it from the exterior. This makes sure that the white pixels of the shape don't come in contact with the white pixels from the outside the black rectangle.

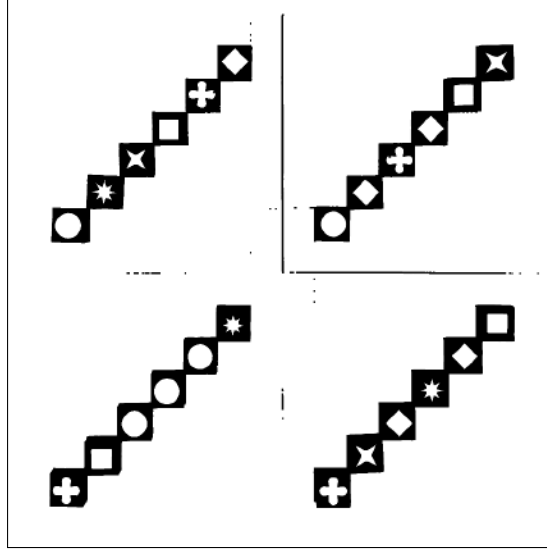


Figure 20: Same mask after applying erosion 4 times with a 3×3 kernel

The contours are filtered by area; anything that doesn't fall within the $[3000, 25000]$ range is discarded. Additionally, anything with a *width* – *height* difference of more than 20 is discarded (so lines and other useless contours are excluded).

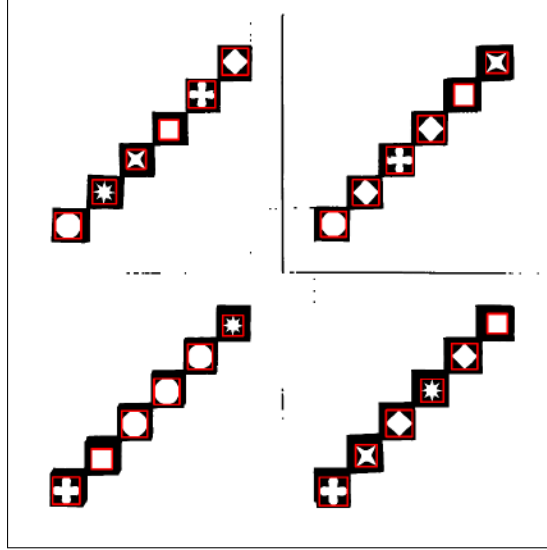


Figure 21: Contours of the shaped pieces

For each contour, the start coordinates and dimensions are obtained and handed as input to the *update_config* function. This function updates the configuration matrix of the board (which is initially an empty 16×16 matrix filled with "0" values) by getting the row, column, and value ("1" or "2" for bonus pieces, shape and color for shaped pieces) of the input piece, storing it in the matrix ($config[row][col] = value$). These are the steps of the function:

1. **Position:** the row and column are obtained by calling the *get_position_from_contour* function, which takes the values of the bounding rectangle of the piece as input and returns the corresponding row and column.
2. **Check for misdetections:** just in case the cell is already occupied by another piece (which shouldn't happen during the initial extraction of the board configuration, but may happen later throughout the game when the contours corresponding to the placed pieces may be too big), a check is performed. If the cell is occupied, the whole process is skipped and nothing is changed.

3. **Increase the contour's dimensions:** make it larger by 50 pixels in all directions with the help of *extend_piece* from *utilities.py*.
4. **Extracting a perfect image of the piece from the source image:** calling *get_piece_outline* from *utilities.py* will return the whole shape.
5. **Color classification:** call *classify_piece_color* from *piece_classification.py*. It iterates through all six colors and tries to extract them from the HSV version of the piece using *cv.inRange*. One of the six resulting masks will contain the greatest amount of white pixels; that mask will correspond to the real color of the piece, which will be returned by the function.
6. **Shape classification:** call *classify_piece_shape* from *piece_classification.py*. First of all, it focuses on the shape of the piece by extracting the black color from the HSV version of the image, negating the result. After that, it resizes the image to match the dimensions of the templates (200x200) by calling *resize_image* from *utilities.py*. It iterates through the six templates, calling *cv.matchTemplate* and ultimately returns the template with the most promising score.
7. **Update:** the row, column, color and shape have been obtained. All that is left to do is update the configuration matrix.

5.1.2 Locating and classifying the number pieces

The number pieces have four distinct properties: the cell has an **orange** border, the number itself also has an **orange** border, the filling of the number is **red**, and the contour areas are **smaller** than the shaped piece areas. To detect the number pieces, the original image is transformed into the HSV color space. Two masks are obtained by calling *cv.inRange* for two ranges of the red color, which are then combined to form one single mask. That mask will contain the red fillings of the numbers, as well as the red shapes.

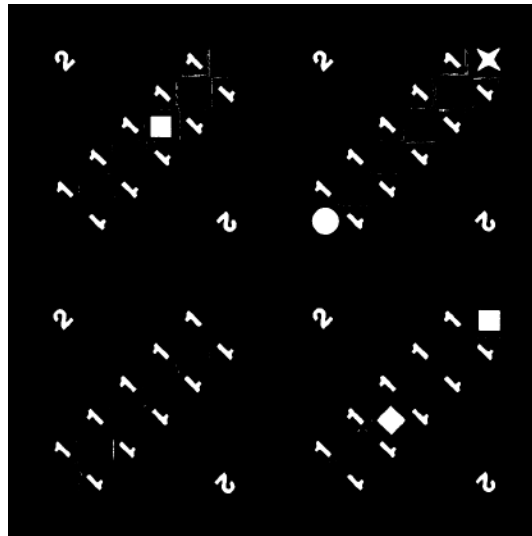


Figure 22: Mask for red colored pieces

The next step is to detect the number pieces and ignore anything else (such as noise or shaped pieces). Picking the contours that have an area within the range [500, 8000] and an absolute *width - height* difference of at most 20 will strictly provide the numbers. If the contour's area is greater than 4000, it is the number "2"; otherwise, it is the number "1". The positions of the number pieces are calculated in the same manner: calling *get_position_from_contour*, inputting the bounding rectangle's values.

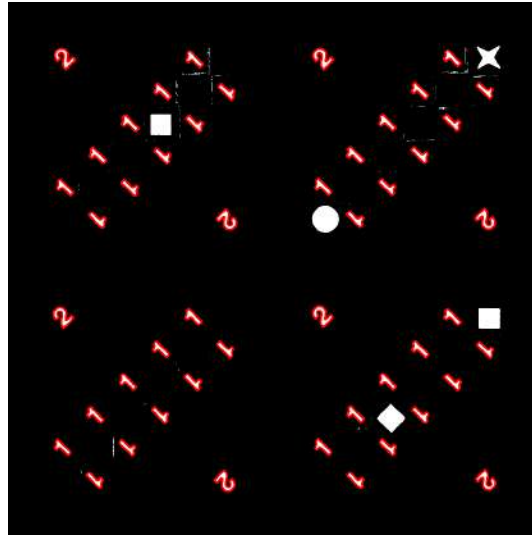


Figure 23: Contours of the number pieces

5.2 Bonus

5.2.1 Locating and classifying the shaped pieces

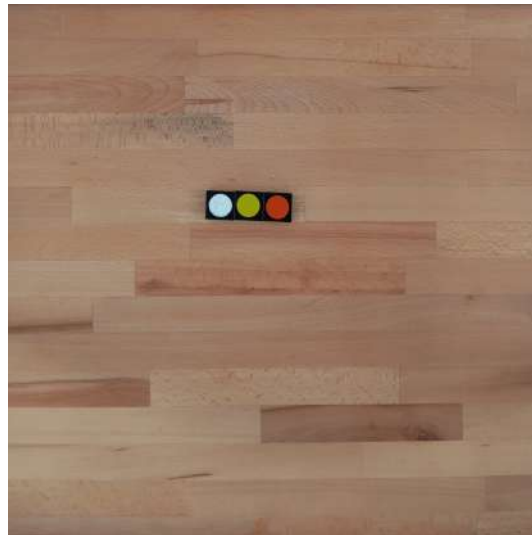


Figure 24: Source image

The image is converted to HSV and then a range is applied to extract the black color. The result is negated, so the emphasis is on the actual shapes.

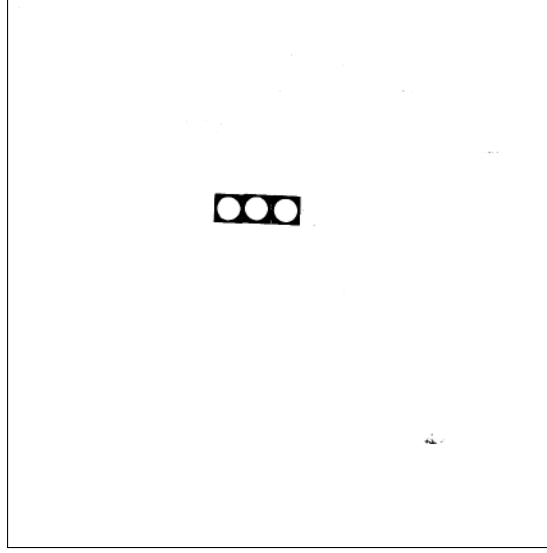


Figure 25: Mask of the shaped pieces

The individual contours of the shapes are obtained. The image is then negated, so the overall bounding rectangle becomes an object of interest. Once again, the contours are obtained, and only the contour of the rectangle is kept.

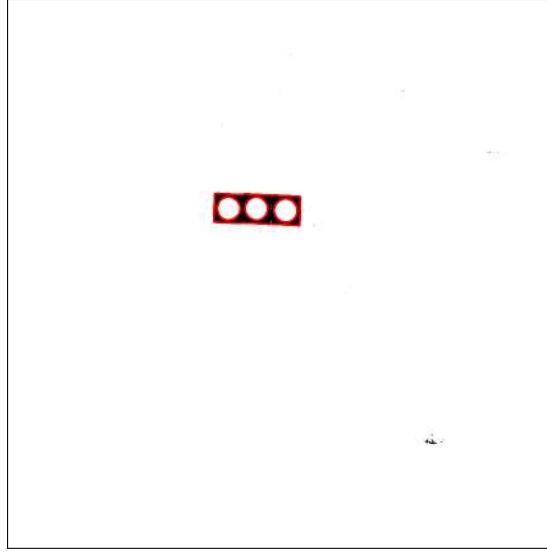


Figure 26: Individual contours and overall contour

The general dimensions of a piece differ from image to image, due to the way the corners are extracted. However, there is no way that the dimensions of a piece should exceed 200×200 . For a 16×16 game-board of the overall dimensions 3072×3072 , a cell would be 192×192 . The extracted boards are 2816×2816 , so a cell should be around 176×176 , although they are usually smaller. The number of placed pieces is given by the amount of individual contours. There are two scenarios:

1. If the width of the overall rectangle exceeds 200, then the pieces are placed in a **horizontal line**. The height of a piece would simply approximately be the height of the rectangle. The final dimensions of a piece are given by the minimum value between the height of the rectangle, and the width of the rectangle divided by the number of pieces.
2. If the height of the overall rectangle exceeds 200, then the pieces are placed in a **vertical line**. The width of a piece would simply approximately be the width of the rectangle. The final dimensions of a piece are given by the minimum value between the width of the rectangle, and the height of the rectangle divided by the number of pieces.

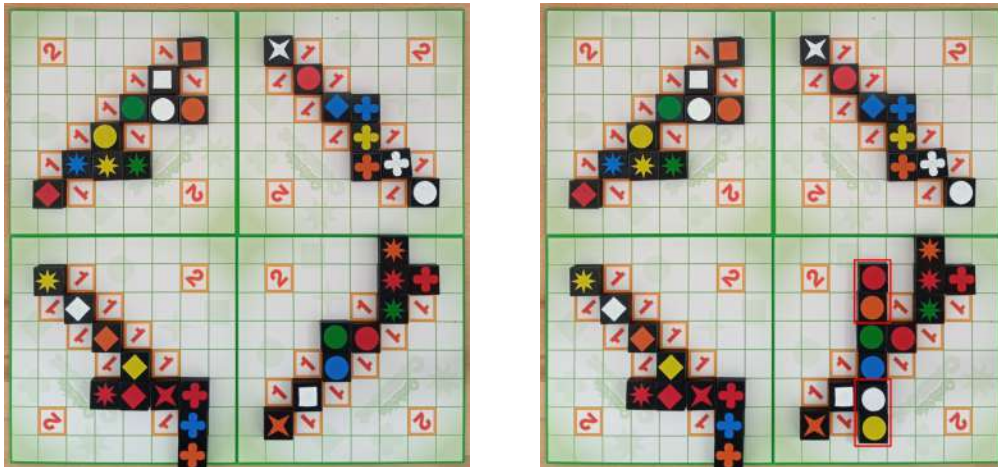
The origin piece is considered to be the leftmost piece if the pieces are placed in a horizontal line, and the highest piece if the pieces are placed vertically. To find it, just get the (x, y) coordinates of the individual contour that minimizes the sum $x + y$ and save them in the global variables *X_ORIGIN* and *Y_ORIGIN*.

The next step is to update the configuration matrix. The function *get_pieces_from_contour* is called, which does the following:

1. **Determine the number of rows and columns:** the number of rows would be the height of the overall rectangle divided by the height of a cell; the result is rounded to an integer using the function *round*. Similarly for the number of columns.
2. **Determine the dimensions of a piece:** earlier we calculated the width and height of a cell. However, those are some general values. Since we are working with a specific contour, we must once again calculate the width and height of a piece, with respect to the contour.
3. **Check for errors:** if either the number of rows or the number of columns is 0, the overall contour is too small (for some reason) and the function stops, returning nothing. If the number of columns and rows are both greater than 1, then the contour is too large and the function stops, returning nothing once again.
4. **Update the configuration matrix:** if *rows* == 1 and *columns* == 1, there was only one piece placed. *update_config* is called, and the piece is added to the config. If *rows* == 1, there are at least two pieces placed horizontally. To iterate through them, start from the *x* coordinate of the overall contour and keep incrementing it by the width of a piece, calculated at *step 2*. If *columns* == 1, the pieces are placed vertically; iterate through them by continuously incrementing with the height of a piece, starting from the *y* coordinate of the overall contour.

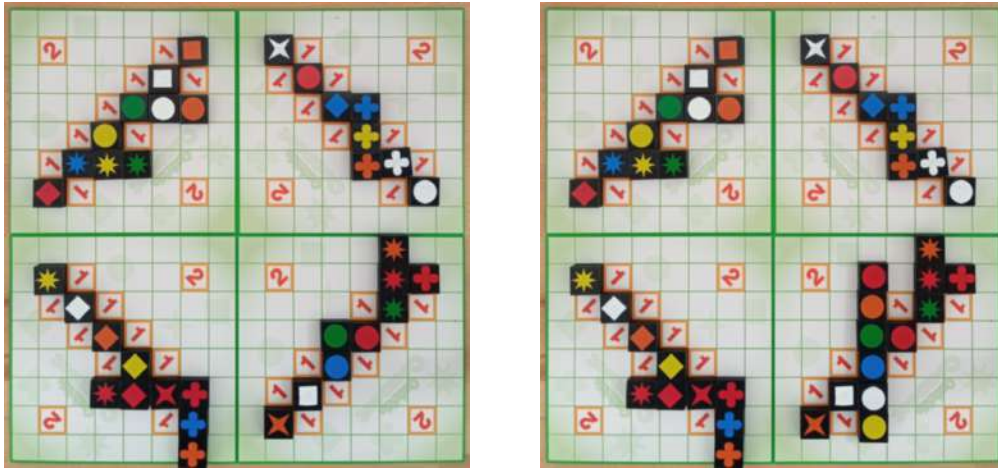
6 Detecting moves

6.1 Connect



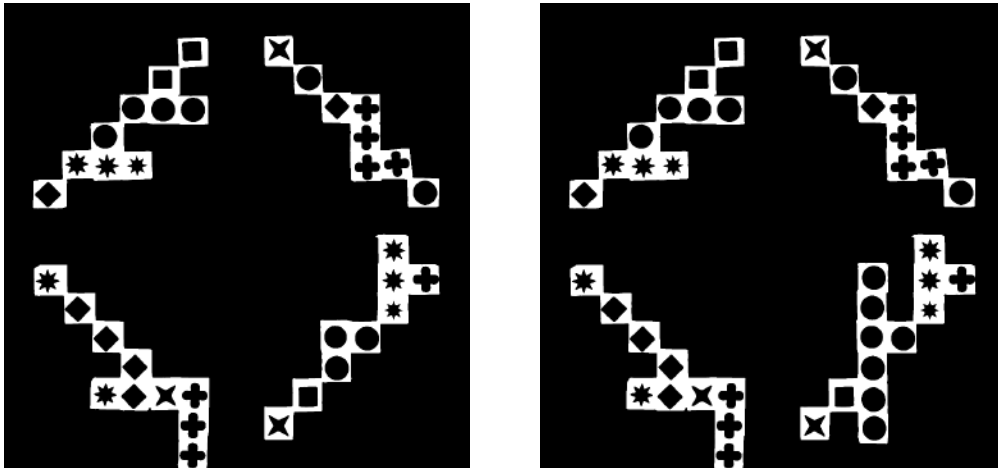
Left image: a board configuration.
Right image: a move has been played.

The two images above are very similar in terms of all aspects. The most noticeable difference is the extra pieces in the right image. Thus, the main idea behind calculating the positions of newly placed pieces would be to compute the difference of the two images and notice which areas could potentially signal the placement of a piece. A starting point would be to apply a **gaussian blur** to both images, smoothing them, making subtle differences have less of an impact.



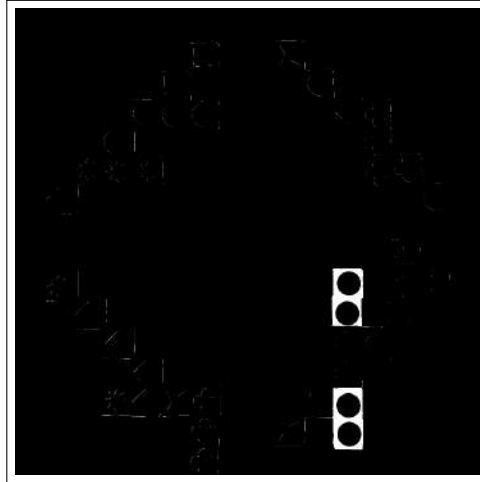
Gaussian blur of size 23×23 applied to both images.

Since we only care about the shaped pieces, two masks are obtained by extracting the black color from the HSV-transformed images.



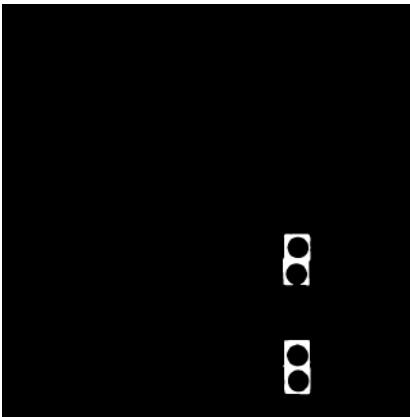
Extracting black from both images

Computing the difference between the masks using *cv.subtract*:

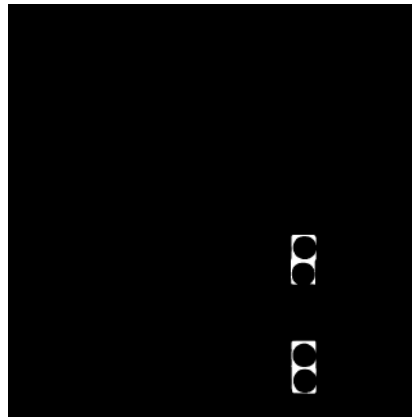


Mask difference

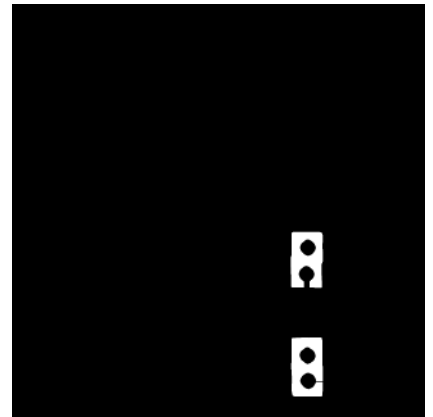
Applying a median blur and eroding the image will get rid of unwanted noise. Dilating the image a bunch of times will make sure that the placed pieces are noticeable, and will help with the contours.



Median blur of kernel size 7.



3 erosions of kernel size 5.



11 dilations of kernel size 5.

The contours that have an area of at least 25000 are selected.

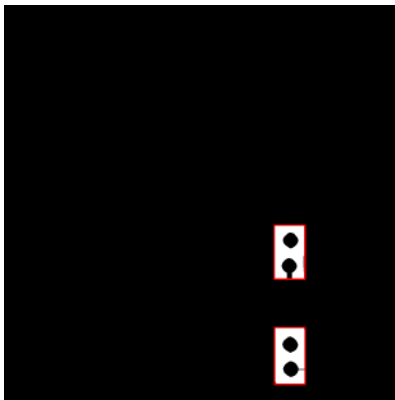


Figure 31: Bounding rectangles of the contours with areas greater than 25000

For each contour, call the *get_pieces_from_contour* function to update the configuration matrix accordingly.

6.2 Bonus

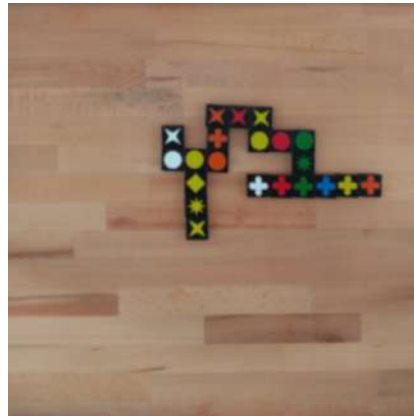
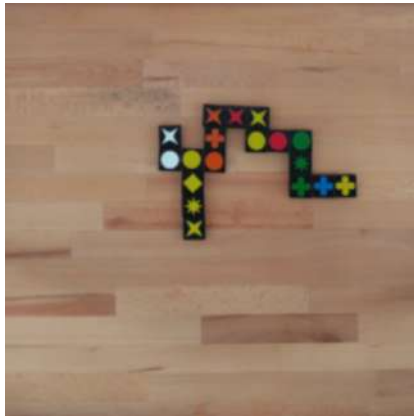


A game configuration.



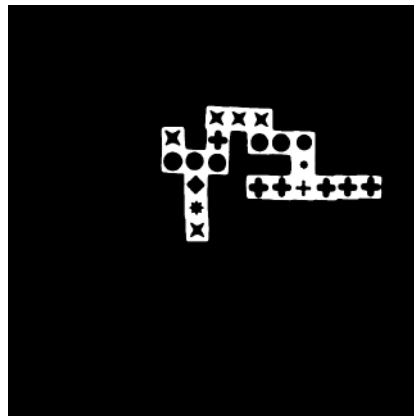
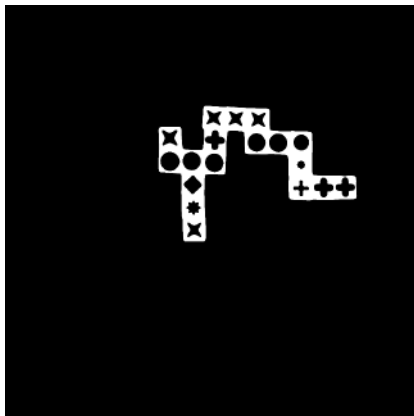
A move has been played.

The idea behind detecting the moves is the same as "Connect". Apply a gaussian blur to the images.



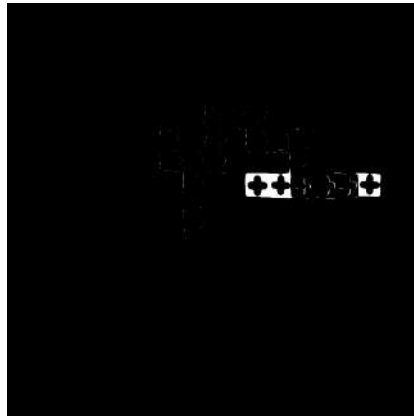
45x45 Gaussian blur applied to both images.

Extract the black color from the HSV-transformed images.



Extracting black from both images

Compute the difference between the two images with *cv.subtract*. Apply a median blur of kernel size 11 to clear out some of the noise.

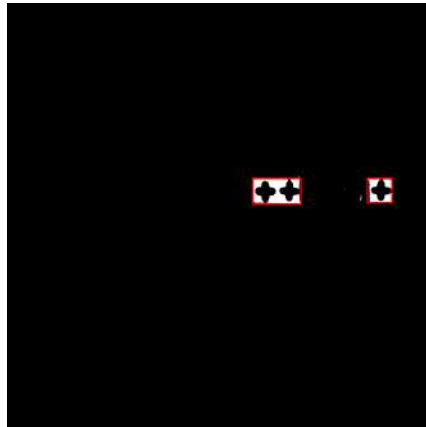


Mask difference.



Apply median blur of kernel size 11.

Find the contours that have an area of at least 6000:



Contours

For each of these contours, call *get_pieces_from_contour*, which updates the configuration.

7 Calculating the score

The score is calculated by calling *get_score* from *calculate_score.py*. The helper functions *up* / *down* / *left* / *right* keep going in the specified direction (e.g., *up* decrements the row position), adding a point for each cell that is not free, returning 0 when out of bounds or when running into an empty cell.

The helper function *leftmost_position* finds (from a given point) the leftmost cell that is not empty (without jumping over any empty cells). The helper function *highest_position* is similar; it finds the highest cell that is not empty (without switching columns and without stopping upon seeing an empty cell).

The helper function *check_full_lines* verifies whether the current move is eligible of the +6 bonus for making a full line.

The steps for computing the total score of a move are as follows:

1. **Single piece:** the score is initialized with the amount of bonus points (0 if it was an empty cell; 1 or 2 otherwise). *leftmost_position* is called in conjunction with *right*, to see how many points are obtained horizontally; *check_full_line* is called to see if the full line bonus applies. Similarly, *highest_position* is called in conjunction with *down*.
2. **Horizontal pieces:** the leftmost piece of the line is found. Starting from that piece and iterating through every single piece of the line to the right, the function *highest_row* is called in conjunction with *down* for each piece that was recently placed. After adding the vertical scores, the horizontal score is calculated and the full line bonus is checked. Bonus points from number pieces are also taken into consideration.
3. **Vertical pieces:** the highest piece of the line is found. Starting from that piece and iterating through every single piece of the line downwards, the function *leftmost_position* is called in conjunction with *right* for each piece that was recently placed. After adding the horizontal scores, the vertical score is calculated and the full line bonus is checked. Bonus points from number pieces are also taken into consideration.